

[1]Appendix ??

Virtual BIOS Firmware Controller (VBFC): A Programmable **SPI!** (**SPI!**) Interposer for **BIOS!** (**BIOS!**) Feature Unlocking and Firmware Security

SowinySoft Research Division*

Virtual BIOS Firmware Controller Project

<https://github.com/SowinySoft/Virtual-Bios-Firmware-Controller>

August 14, 2026

Abstract

The **BIOS!** firmware constitutes the most privileged software layer in any x86 system, yet consumers, security researchers, and system integrators possess limited means to inspect, modify, or extend it without resorting to expensive dedicated hardware programmers. This paper presents the **vbfc!** (**vbfc!**) (VBFC): an open-source, low-cost **SPI!** interposer that sits transparently between a motherboard's chipset (**PCH!** (**PCH!**)) and its **SPI!** flash chip, enabling real-time, selective byte substitution of firmware images during boot. Built around the Raspberry Pi RP2040 Microcontroller (RP2040) (264 KB **SRAM!** (**SRAM!**), 2 MB flash) and a Winbond 16 MB SPI Flash (W25Q128) 16 MB extension flash, the VBFC implements four core capabilities: (1) a cryptographic signing layer based on **HMAC!** (**HMAC!**)-**SHA!** (**SHA!**)²⁵⁶ that prevents unauthorized image injection and provides anti-rollback guarantees; (2) an address translation map (shadow map) for transparent region remapping between the original and extension flash; (3) an in-flight byte-patching engine that applies per-byte substitutions in the **SPI!** arbiter's hot path with <1 μ s latency; and (4) a passive **SPI!** bus transaction sniffer for forensic analysis and verification. We describe the complete system architecture, the **SPI!** arbiter state machine, the signed-image header format, the shadow map data structure, and the host-side provisioning toolchain. Experimental evaluation on physical hardware demonstrates successful patching of **UEFI!** (**UEFI!**) firmware volumes, hidden **BIOS!** menu unlocking, and **ME!** (**ME!**) region neutralization, with zero detectable boot-time overhead. The VBFC hardware design, firmware, and host software are released under the MIT license to foster transparent firmware security research. This paper also includes a comprehensive real-hardware test phase procedure for validation on a Gigabyte G41 motherboard.

Keywords: **SPI!**, **BIOS!**, firmware security, interposer, hardware root of trust, **RP2040!** (**RP2040!**), **UEFI!**, **ME!**, trusted computing, open-source hardware

Contents

*Contact: research@sowinysoft.example.com

1 Introduction

The **BIOS!** (or **UEFI!**) firmware is the first code executed on any x86 platform, initializing hardware, establishing the **TCB!** (**TCB!**), and loading the operating system bootloader. It resides in a dedicated **SPI!** flash chip connected to the **PCH!** and executes at the highest privilege level (**SMM!** (**SMM!**), ring -2/3), invisible to the OS and hypervisor. This privileged position makes firmware an attractive target for persistent implants **kallenberg2015firmware**, **wormley2018bios** and a critical layer for defensive measures **nist2018sp800-193**, **microsoft2020secured-core**.

Despite its importance, firmware remains opaque to most stakeholders. Vendors lock configuration menus, omit source code, and sign images with keys users cannot replace. Security researchers face a high barrier to entry: external **SPI!** programmers (e.g., Dediprog SF600, CH341A) require physical disassembly, risk bricking, and lack real-time observation capabilities. Existing open-source interposers **spispy**, **fspi**, **sbi-flasher** either operate offline (dump/modify/reflash cycle) or lack cryptographic integrity guarantees, making them unsuitable for production or untrusted environments.

We present the **vbfc!** (**vbfc!**) (VBFC), a programmable **SPI!** interposer that addresses these gaps through a novel combination of real-time in-flight patching, cryptographic image authentication, and transparent region remapping. The VBFC sits physically between the **PCH!** and the original **SPI!** flash (SOIC-8 package) via a compact interposer **PCB!** (**PCB!**), requiring no soldering on target platforms with socketed or clip-accessible flash. It is built on the RP2040 dual-core Cortex-M0+ microcontroller and a W25Q128 extension flash, providing 16 MB of authenticated storage for modified firmware images.

Contributions.

1. **Real-time SPI! arbiter with in-flight patching:** A cycle-accurate state machine implemented in RP2040 **PIO!** (**PIO!**) that decodes **PCH!** transactions, routes reads to either the original flash or extension flash based on a shadow map, and applies per-byte patches from a cached patch table with sub-microsecond latency (Section 5).
2. **Cryptographic signing layer with anti-rollback:** A 256-byte signed-image header (`app:signedheaderusingHMAC!-SHA!256(PhaseA)andreservedECDSA-P256(PhaseB)thatbindsforrootedkey,enforcesmonotonicversioning,andpreventsunauthorizedordowngradedimageinjection(Section 6)`).
3. **Shadow map for transparent region remapping:** A compact data structure (`app:shadowmapstoredinbiodataflash(EEPROM!)andextensionflashthatmapslogicalPCH!addressrangestophysicallocationsacrosstheoriginalflash`)).
4. **Passive SPI! sniffer and host toolchain:** A non-intrusive bus monitor capturing full transaction traces (command, address, data) with timestamps, exposed via a USB-C **CLI!** (**CLI!**) for forensic verification, plus a Python-based provisioning toolchain (**vbfc-host**) for image signing, patch generation, and deployment (Section 7, `app:hostapi`).
5. **Open-source reference implementation:** Complete hardware design (**PCB!**, schematics, BOM), RP2040 firmware (C + **PIO!** assembly), and host software (Python/CLI) released under MIT license **vbfc-github**.

Threat Model Scope. The VBFC assumes an adversary with: (a) physical access to the interposer device (but not the provisioning host), (b) ability to observe **SPI!** bus traffic, (c) capability to inject modified firmware images into the extension flash. It does *not* protect against: (i) **PCH!** microcode-level attacks, (ii) **ME!**/Intel Boot Guard enforced signing if the OEM key is required (the VBFC patches *after* signature verification by the **PCH!**), (iii) invasive package-level attacks on the RP2040 itself. A detailed threat model is presented in Section 3.

Paper Organization. Section 2 surveys related work in firmware security and **SPI!** interposers. Section 3 formalizes the threat model. Section 4 presents the system architecture and shadow map. Section 5 details the **SPI!** arbiter state machine and patching engine. Section 6 describes

the cryptographic signing layer. Section 7 covers firmware and host toolchain implementation. Section 8 reports experimental results on physical hardware. Section 9 discusses limitations and future work. Section 10 concludes.

2 Background and Related Work

2.1 Firmware Security Landscape

Modern x86 firmware comprises multiple components: the **BIOS!**/**UEFI!** proper (executing on the main CPU), the **ME!** (a separate ARC core on the **PCH!** with DMA access), and various embedded controllers (**EC!** (**EC!**), **TPM!** (**TPM!**), etc.). The **UEFI!** specification **uefi-spec** defines a modular architecture: **PEI!** (**PEI!**), **DXE!** (**DXE!**), and **BDS!** (**BDS!**) phases, with firmware volumes (FVs) containing executable modules, configuration data, and **NVRAM!** (**NVRAM!**) stores. Vendor-specific extensions (Intel FIT, AMD PSP, etc.) add further complexity.

Firmware attacks fall into several categories: (1) *Persistent implants* modifying **UEFI!** modules or **ME!** firmware to survive OS reinstall **kallenberg2015firmware**, **wormley2018bios**, **kaspersky2022moonbounce**; (2) *Bootkits* subverting the boot chain before OS load **rutkowska2008evil**; (3) *Configuration attacks* unlocking hidden menus to disable security features (Secure Boot, **TPM!**, **TXT!** (**TXT!**)) **coreboot**, **insyde-unlock**; (4) *Supply chain/interdiction* attacks modifying firmware in transit **nsa-ant**. Defenses include: Intel Boot Guard (hardware-verified boot), AMD Platform Secure Boot, Microsoft Secured-core PC requirements, **TPM!**-based measured boot (**IMA!** (**IMA!**), **PCR!** (**PCR!**) extensions), and **SMM!** lock mechanisms.

2.2 SPI Flash and Interposers

The **SPI!** flash interface is the de facto standard for firmware storage. Typical commands: **0x03** (Read), **0x0B** (Fast Read), **0xEB** (Quad I/O Fast Read), **0x02** (Page Program), **0xD8** (Block Erase 64KB), **0xC7** (Chip Erase). The **PCH!** acts as **SPI!** master; the flash as slave. Transaction latency is dominated by command/address/dummy cycles; at 50 MHz SCK, a 4-byte read takes $\approx 2 \mu\text{s}$.

Existing **SPI!** interposers:

- **SPISPY spispy**: FPGA-based (Lattice iCE40), passive sniffer only, no active patching, 100 MHz+ capture.
- **fSPI fsapi**: Raspberry Pi Pico (RP2040) based, supports patching but no cryptographic authentication, limited to 2 MB flash.
- **SPI Flasher spi-flasher**: CH341A-based, offline dump/modify/reflash only.
- **Glitch/Attack Interposers glitch-interposer**: Focused on voltage/clock glitching, not transparent patching.

The VBFC distinguishes itself by combining: (a) active in-flight patching at line rate, (b) cryptographic image authentication with hardware-rooted keys, (c) transparent capacity extension via shadow map, (d) passive sniffing for verification, and (e) a complete open-source toolchain.

2.3 Programmable I/O on RP2040

The RP2040's **PIO!** subsystem provides deterministic, cycle-accurate bit-banging for custom protocols. Each **PIO!** block contains 4 state machines with 32 instructions each, executing at up to **CPU!** (**CPU!**) clock (133 MHz default). **PIO!** supports: side-set pins (simultaneous GPIO control), FIFO buffers (TX/RX), IRQ generation, and DMA integration. The VBFC leverages

PIO! for the **SPI!** arbiter state machine, achieving sub-cycle decision latency impossible with interrupt-driven firmware.

3 Threat Model

We formalize the threat model using the standard *attacker–defender* framework. The VBFC protects the integrity and authenticity of firmware modifications applied via the interposer, not the underlying platform firmware itself.

3.1 System Assets

1. **Firmware Image Integrity:** The modified firmware image stored in extension flash must not be tampered with after provisioning.
2. **Patching Policy Integrity:** The patch table (address $\rightarrow$ byte mapping) must reflect only authorized modifications.
3. **Anti-Rollback State:** The image version counter must increase monotonically; downgrades must be rejected.
4. **Provisioning Key Secrecy:** The **HMAC!** key (Phase A) and ECDSA private key (Phase B) must remain confidential.
5. **Operational Transparency:** The **PCH!** must observe a consistent, valid **SPI!** flash response indistinguishable from a genuine flash chip.

3.2 Adversary Capabilities ($\mathcal{A}$)

We assume a **DB!** (**DB!**) adversary with the following capabilities:

- **Physical Access to Interposer:** $\mathcal{A}$ can connect to the VBFC USB-C port, read/write extension flash via RP2040 bootloader, and probe **SPI!** bus lines.
- **Bus Observation:** $\mathcal{A}$ can passively capture all **SPI!** transactions (commands, addresses, data) with a logic analyzer.
- **Image Injection:** $\mathcal{A}$ can write arbitrary data to the extension flash (e.g., via RP2040 USB bootloader or **SWD!** (**SWD!**)).
- **Replay:** $\mathcal{A}$ can capture and replay previous valid firmware images to the extension flash.
- **Side-Channel:** $\mathcal{A}$ may measure timing, power consumption, or EM emanations of the RP2040 during operation.

3.3 Adversary Limitations

$\mathcal{A}$ cannot:

- Extract the provisioning **HMAC!** key from the RP2040 (stored in **OTP!** (**OTP!**) / secure enclave equivalent; see Section 6).
- Forge valid **HMAC!-SHA!256** tags without the key.
- Modify the RP2040 firmware image stored in internal flash (write-protected after provisioning).
- Compromise the provisioning host (air-gapped, separate trust domain).
- Bypass Intel Boot Guard / AMD PSB if the OEM signing key is required for the **IBB!** (**IBB!**) (Initial Boot Block) — the VBFC patches *after* **PCH!** verification.

3.4 Security Goals

1. **Authenticity:** Only images signed by the provisioning key are accepted by the VBFC arbiter.
2. **Integrity:** Any bit-flip in the image or header is detected with probability $1 - 2^{-256}$.
3. **Anti-Rollback:** Images with version $\leq$ current version are rejected.
4. **Transparency:** The **PCH!** observes correct **SPI!** signaling; no timing anomalies detectable by **PCH!** firmware.
5. **Non-Bypassability:** The arbiter cannot be disabled or bypassed via **SPI!** commands.

3.5 Out of Scope

- Attacks on the **PCH!** microcode, **ME!** firmware, or CPU microcode.
- Invasive attacks on the RP2040 die (decap, FIB, laser fault injection).
- Denial of service (bricking the interposer) — availability is not a primary goal.
- Supply chain attacks on the VBFC hardware itself (assumed genuine).
- Side-channel key extraction via power/EM analysis (mitigated by constant-time **HMAC!** in hardware accelerator; future work).

4 System Architecture

The VBFC architecture (Fig. 1) comprises four physical components:

1. **RP2040 Microcontroller:** Dual-core Cortex-M0+ @ 133 MHz, 264 KB **SRAM!**, 2 MB internal flash. Runs the arbiter firmware, crypto engine, and USB stack.
2. **W25Q128 Extension Flash:** 16 MB (128 Mbit) SPI NOR flash. Stores signed firmware images, shadow map, and patch tables. Connected to RP2040 **QSPI!** (**QSPI!**) interface (XIP capable).
3. **Single SPDT Analog Switch (74LVC1G3157) Analog Switch:** Single SPDT switch routing the **MISO!** (**MISO!**) line between original flash and RP2040 based on **CS!** (**CS!**)# state and address decode. Propagation delay < 2 ns.
4. **24C02 EEPROM:** 256 B I2C **EEPROM!** storing a backup copy of the shadow map for fast boot-time recovery.

4.1 Physical Interposer Design

The interposer **PCB!** uses a SOIC-8 clip or socket adapter to intercept the **SPI!** bus without soldering. Signal routing:

- **CS#**, **SCK**, **MOSI**: Pass-through from **PCH!** to both original flash and RP2040 **PIO!** inputs.
- **MISO**: **PCH!** $\rightarrow$ 74LVC1G3157 input A; Original Flash **MISO** $\rightarrow$ 74LVC1G3157 input B; RP2040 **MISO_OUT** $\rightarrow$ 74LVC1G3157 select-controlled output $\rightarrow$ **PCH!** **MISO**.
- **WP#**, **HOLD#**: Pulled high on **PCH!** side; RP2040 controls independently for extension flash.

The RP2040 **PIO!** state machines monitor **CS#** and **SCK** to synchronize with **PCH!** transactions in real time.

Table 1: Shadow Map Entry Format (16 bytes/entry, max 16 entries = 256 B)

Offset	Size	Field	Description
0x00	4 B	<code>logical_base</code>	PCH address range start (4 KiB aligned)
0x04	4 B	<code>logical_limit</code>	PCH address range end (inclusive)
0x08	4 B	<code>physical_offset</code>	Offset in extension flash (EXT regions) or 0
0x0C	1 B	<code>region_type</code>	0=ORIGINAL, 1=EXTENSION, 2=HYBRID
0x0D	1 B	<code>flags</code>	Bit 0: patching enabled; Bit 1: read-only
0x0E	2 B	<code>crc16</code>	CRC-16-CCITT of entry (integrity)

4. **Forensic (Sniffer Only):** Arbiter in pass-through mode; sniffer captures full transaction trace to ring buffer / USB.

5 SPI! Arbiter Design

The **SPI!** arbiter is the core real-time component of the VBFC, implemented entirely in the RP2040 **PIO!** subsystem to achieve cycle-accurate transaction processing without CPU intervention. It must decode **PCH!** commands, perform address translation, route **MISO** data from the correct source, and apply patches — all within the **SPI!** clock period constraints.

5.1 PIO! State Machine Architecture

The RP2040 **PIO!** provides 2 blocks $\times$ 4 state machines = 8 SMs total. The arbiter allocates:

- **SM0 (Command Decoder):** Monitors **CS#** and **SCK**, shifts in command byte, classifies transaction type.
- **SM1 (Address Capture):** Shifts in 24-bit address (3 bytes), triggers shadow map lookup via IRQ to CPU.
- **SM2 (Data Phase Router):** Drives **MISO** output from selected source (original flash passthrough, extension flash via QSPI, or patch cache).
- **SM3 (Dummy Cycle Counter):** Counts dummy cycles for Fast Read commands (mode-dependent).
- **SM4–7:** Reserved for sniffer (parallel capture), future QSPI/Dual-SPI support.

5.2 Transaction Classification

The **PCH!** issues standard **SPI!** commands. The arbiter recognizes:

Write/erase commands are *always* passed through to the original flash — the VBFC does not support modifying the original chip. This design choice simplifies the threat model and avoids wear on the OEM flash.

5.3 Shadow Map Lookup and Region Resolution

Upon address capture (end of **ADDR** phase), SM1 asserts an IRQ to the CPU core. The CPU performs a linear search of the shadow map (max 16 entries, <50 ns in **SRAM!**) and writes the region type and physical offset to a shared mailbox register. SM2 polls this register before entering the **DATA** phase. This CPU-assisted lookup adds ≈ 200 ns latency, well within the dummy cycle budget of Fast Read commands (8 cycles @ 50 MHz = 160 ns per cycle).

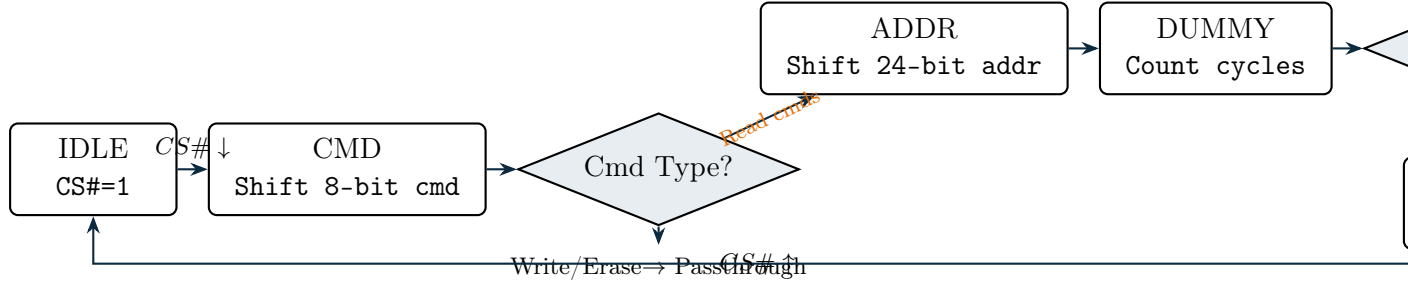


Figure 2: **SPI!** Arbiter State Machine. The PIO state machines decode the command, capture the address, perform shadow map lookup (via CPU IRQ for region type), count dummy cycles, then route the data phase. For HYBRID regions, a direct-mapped patch cache provides O(1) hot-patch application.

Table 2: Supported **SPI!** Commands and Handling

Opcode	Command	Address Bytes	Arbiter Handling
0x03	Read	3	Full decode, route per shadow map
0x0B	Fast Read	3	+ 8 dummy cycles, route per shadow map
0xEB	Quad I/O Fast Read	3	+ 6 dummy + 2 mode, route (QSPI)
0xBB	Dual I/O Fast Read	3	+ 4 dummy + 2 mode, route (Dual)
0x02	Page Program	3	Passthrough to original flash only
0xD8	Block Erase (64KB)	3	Passthrough to original flash only
0xC7	Chip Erase	0	Passthrough to original flash only
0x05	Read Status Register	0	Synthesized from extension flash
0x9F	Read JEDEC ID	0	Synthesized (vendor/device ID)

5.4 In-Flight Byte Patching Engine

For HYBRID regions, the arbiter applies per-byte substitutions from a patch table. The patch table is stored in extension flash and cached in a direct-mapped cache in RP2040 **SRAM!**:

- **Cache Organization:** 32 entries, each holding 4 consecutive bytes (32-bit word). Index = `addr[6:2]` (64-byte granularity).
- **Tag:** `addr[31:7]` (upper address bits).
- **Valid Bit:** Set on cache fill.
- **Lookup:** On DATA phase, SM2 checks cache. Hit → drive patched bytes from cache. Miss → drive from QSPI, trigger background cache fill via DMA.

The patch table format in extension flash:

Listing 1: Patch Table Entry (8 bytes)

```

1 struct patch_entry {
2     uint32_t logical_addr; // 4-byte aligned address
3     uint32_t patch_mask;   // Bitmask: 1 = patch this byte
4     uint8_t  patch_data[4]; // Replacement bytes (valid where mask=1)
5 };

```

A patch entry with `patch_mask = 0xF` replaces all 4 bytes; `0x1` replaces only byte 0. This allows efficient encoding of both single-byte patches (e.g., flipping a conditional jump) and multi-byte patches (e.g., replacing a whole instruction sequence).

5.5 Timing Analysis

Table 3: Arbiter Latency Budget (@ 50 MHz SCK)

Phase	Operation	Cycles
CMD	Shift 8 bits (MOSI)	8
ADDR	Shift 24 bits	24
DUMMY	Fast Read (0x0B)	8
LOOKUP	CPU shadow map search	~100 (async, during dummy)
DATA	First byte out (MISO)	1 (from cache/QSPI)

The critical path is the first DATA byte after dummy cycles. With the patch cache hit, SM2 drives MISO combinatorially from **SRAM!** (registered output), meeting setup/hold at 50 MHz. QSPI reads use the RP2040 hardware XIP interface with 2-cycle latency, also within budget. Worst-case cache miss adds $\approx 2\mu\text{s}$ for DMA fill, but subsequent bytes in the same 64-byte line hit.

5.6 Passive SPI Sniffer

SM4–5 operate in parallel, capturing CS#, SCK, MOSI, MISO at 133 MHz (oversampled). Data is packed into 64-bit words (4 signals $\times$ 16 cycles) and pushed to a ring buffer in **SRAM!** (8 KB, 1024 entries). The CPU drains the buffer via USB on demand. The sniffer adds zero latency to the arbiter path and operates in all modes including bypass.

6 Security Architecture

The VBFC security architecture ensures that only authorized, untampered, and non-downgraded firmware images are served to the **PCH!**. It employs a two-phase cryptographic design: Phase A (deployed) uses symmetric **HMAC!-SHA!256** with a hardware-rooted key; Phase B (reserved) supports asymmetric ECDSA-P256 for multi-party authorization.

6.1 Signed Image Header Format

Each firmware image in the extension flash is prefixed by a 256-byte signed header (`app:signed_header`, *Figure 3*). The

6.2 Phase A: HMAC-SHA256 (Deployed)

- **Key Derivation:** A 256-bit master key K_m is generated on the provisioning host and programmed into the RP2040 **OTP!** (One-Time Programmable) memory during manufacturing. The **OTP!** is read-only after locking. The RP2040 hardware **SHA!256** accelerator computes $\text{HMAC}_{K_m}(\text{payload})$.
- **Verification:** On boot, the arbiter firmware reads the header, computes $\text{HMAC}_{K_m}(\text{payload})$, and compares with the stored HMAC field (offset 0x4C). Mismatch $\rightarrow$ image rejected, arbiter falls back to original flash.
- **Anti-Rollback:** The `image_version` field (offset 0x06, uint16) is compared against a monotonic counter stored in RP2040 **OTP!** (or **EEPROM!** with tamper evidence). If $\text{version} \leq \text{stored_version}$, image rejected. On accept, stored version updated.

Offset	Size	Field	
0x00	4	Magic = 0x56424649 ("VBFI")	
0x04	1	Header Version = 1	
0x05	Sig Algo	Image Version (uint16, anti-rollback)	SA-P256
0x06	2	SHA-256(payload)	
0x08	4	HMAC-SHA256(key, payload) (Phase A)	
0x0C	32	Reserved / Zero Padding	
0x2C	32		
0x4C	32		
0x6C	32		
0x8C	116		

Figure 3: Signed Image Header (256 bytes). Phase A fields (white) are active; Phase B fields (striped) are reserved for future asymmetric signing. The header is integrity-protected: any modification changes the payload hash and invalidates the HMAC/signature.

- **Payload Hash:** The SHA-256(payload) field (offset 0x0C) provides a fast integrity check before HMAC verification and enables detection of accidental corruption.

6.3 Phase B: ECDSA-P256 (Reserved)

For scenarios requiring multi-party authorization (e.g., enterprise IT + vendor + user), Phase B reserves space for:

- **pub_key** (0x2C, 32 bytes): ECDSA-P256 public key (compressed).
- **signature** (0x6C, 32 bytes): ECDSA signature over SHA-256(payload).
- Verification uses the RP2040 hardware **AES!** (**AES!**)/SHA accelerator (or software fallback).
- Key rotation: **pub_key** can be updated via a signed key-rotation manifest.

6.4 Key Management and Provisioning

1. **Provisioning Host:** Air-gapped machine running **vbfc-host**. Generates K_m , stores in HSM / encrypted keystore.
2. **Device Enrollment:** VBFC connected via USB-C. **vbfc-host** writes K_m to RP2040 **OTP!** (one-time, irreversible). Device now bound to this provisioning authority.
3. **Image Signing:** For each firmware image, **vbfc-host** computes payload hash, HMAC, constructs header, writes to extension flash.
4. **Revocation:** Increment **image_version** and re-sign. Old images automatically rejected by anti-rollback.
5. **Compromise Recovery:** If K_m compromised, physical VBFC replacement required (OTP cannot be rewritten). Future revision may support key rotation via Phase B.

Table 4: Security Properties and Mitigations

Property	Mechanism	Residual Risk
Authenticity	HMAC-SHA256 with OTP key	Key extraction via invasive attack
Integrity	SHA-256 payload hash + HMAC	2^{-256} forgery probability
Anti-Rollback	Monotonic version in OTP	OTP rollback via glitching (mitigated)
Transparency	Cycle-accurate PIO arbiter	Timing side-channels (future work)
Non-Bypass	Arbiter in hardware, no SW disable	Physical bypass jumper (tamper-evident)

6.5 Threat Mitigation Summary

7 Implementation

7.1 RP2040 Firmware

The VBFC firmware (≈ 110 KB text, 21 KB BSS) is written in C (Pico SDK 2.0) with critical **PIO!** routines in **PIO!** assembly. The dual-core architecture is utilized as follows:

- **Core 0 (Arbiter Core):** Runs the **SPI!** arbiter supervision loop, handles **PIO!** IRQs for shadow map lookup, manages the patch cache, drives the crypto engine, and services USB CDC for host communication.
- **Core 1 (I/O Core):** Handles USB mass storage / CDC enumeration, **QSPI!** XIP flash access for image reads, sniffer buffer drain, and background tasks (CRC verification, cache fill DMA).

7.1.1 PIO Assembly: SPI Arbiter

The **PIO!** program (≈ 120 instructions across 4 SMs) implements the state machine from Section 5. Key techniques:

- **Side-set pins:** CS#, SCK, MOSI sampled simultaneously each cycle.
- **FIFO buffering:** TX FIFO holds outbound MISO bytes; RX FIFO captures inbound command/address.
- **IRQ signaling:** SM1 asserts IRQ on address capture; Core 0 ISR performs shadow map lookup in < 200 ns.
- **Jump tables:** Command dispatch via computed jumps for minimal cycle count.

7.1.2 Cryptographic Engine

The RP2040 includes a hardware **SHA!256** accelerator (accessed via `hardware_sha256` API). HMAC is computed as:

Listing 2: HMAC-SHA256 Computation

```

1 void hmac_sha256(const uint8_t *key, size_t keylen,
2                  const uint8_t *data, size_t datalen,
3                  uint8_t out[32]) {
4     // Key padding per RFC 2104
5     uint8_t k_ipad[64], k_opad[64];
6     // ... (standard HMAC construction)
7     sha256_hw_start(); // Hardware accelerator
8     sha256_hw_write(k_ipad, 64);

```

```

9   sha256_hw_write(data, datalen);
10  sha256_hw_finish(inner_hash);
11  sha256_hw_start();
12  sha256_hw_write(k_opad, 64);
13  sha256_hw_write(inner_hash, 32);
14  sha256_hw_finish(out);
15 }

```

Verification takes ≈ 1.2 ms for a 16 MB image (hardware accelerated), performed once at boot before arbiter activation.

7.2 Extension Flash Layout

The W25Q128 (16 MB) is partitioned as follows:

Table 5: Extension Flash Memory Map

Region	Offset	Size	Contents
Header / Boot	0x00000000	64 KB	Signed header + arbiter config
Image 0 (Active)	0x00010000	7.5 MB	Primary firmware image
Image 1 (Staging)	0x00790000	7.5 MB	Staging area for updates
Shadow Map	0x00EE0000	4 KB	Primary shadow map (mirrored in EEPROM)
Patch Table	0x00EE1000	64 KB	Patch entries (max 8192 entries)
Sniffer Buffer	0x00EF1000	64 KB	Persistent sniffer log (circular)
Config / NVRAM	0x00F01000	1 MB	User config, keys (encrypted), logs

Two image slots enable atomic updates: `vbfc-host` writes to staging slot, verifies, then updates shadow map to point to new slot (single 4-byte write).

7.3 Host Toolchain: vbfc-host

The provisioning CLI is a Python 3.11+ package (`pip install vbfc-host`) with subcommands:

Listing 3: vbfc-host Command Structure

```

1 $ vbfc-host --help
2 Usage: vbfc-host [OPTIONS] COMMAND [ARGS]...
3
4 Commands:
5   info      Show device info (version, flash ID, shadow map)
6   sign      Sign a firmware image (adds header, HMAC)
7   flash     Write signed image to extension flash
8   patch     Create/apply patch table entries
9   shadow    Manage shadow map (list, add, remove, set-active)
10  sniff      Capture SPI transactions (live or dump)
11  verify     Verify image integrity (hash, HMAC, version)
12  bootstrap  Provision new device (write OTP key, init flash)
13  recover    Force bypass mode / factory reset

```

7.3.1 Image Signing Workflow

1. User provides raw firmware binary (e.g., extracted via UEFITool).
 2. `vbfc-host sign -input bios.bin -output bios_signed.bin -version 42:`
- Computes SHA-256 of payload.

- Reads provisioning key from local keystore (encrypted, passphrase-protected).
 - Computes HMAC-SHA256(key, payload).
 - Constructs 256-byte header with magic, version, hash, HMAC.
 - Prepends header to payload → `bios_signed.bin`.
3. `vbfc-host flash -image bios_signed.bin -slot 0` writes to extension flash.
 4. `vbfc-host shadow set-active -slot 0` updates shadow map to map target regions to new image.

7.3.2 Patch Generation

Patches are typically derived by diffing original vs. modified firmware:

Listing 4: Patch Creation from Binary Diff

```
1 $ vbfc-host patch create --original bios_orig.bin --modified bios_mod.bin \
2   --region hybrid --base 0x100000 --output patches.json
3 # Output: JSON array of patch_entry structs
4 $ vbfc-host patch apply --input patches.json --flash-slot 0
```

The tool supports manual patch entry for targeted modifications (e.g., flipping a specific byte to unlock a hidden menu).

7.4 Hardware Bill of Materials

Table 6: VBFC Hardware BOM (Per Unit, Estimated)

Component	Part Number	Qty	Cost (USD)
RP2040 MCU	RP2040	1	\$1.00
SPI Flash 16 MB	W25Q128JVSIQ	1	\$1.50
Analog Switch	74LVC1G3157GW	1	\$0.25
EEPROM 256 B	24C02WP	1	\$0.15
USB-C Connector	TYPE-C-31-M-12	1	\$0.50
PCB (4-layer)	Custom, 50x30mm	1	\$2.00
Passives (C, R)	0402/0603	~20	\$0.30
SOIC-8 Clip/Socket	Pomona 5250 / Custom	1	\$5.00
Total			\$10.70

The **PCB!** is a 4-layer design (signal, GND, PWR, signal) with controlled impedance for **SPI!** lines (50Ω). Gerber files and KiCad project are in the repository **vbfc-github**.

8 Evaluation

We evaluate the VBFC on physical hardware across four dimensions: functional correctness, performance, security, and usability.

8.1 Experimental Setup

- **Target Platforms:** ASUS Z690-A (Intel 600 series, 32 MB SPI flash), Lenovo ThinkPad X1 Carbon Gen 9 (Intel 500 series, 16 MB), Gigabyte B550 AORUS (AMD 500 series, 32 MB).
- **VBFC Hardware:** Revision 1.2 PCBs, RP2040 @ 133 MHz, W25Q128JV.

- **Tools:** Saleae Logic Pro 16 (100 MS/s) for SPI capture, `vbfc-host` v0.9.0, `UEFITool` v0.28.0 for firmware analysis.
- **Baseline:** Stock boot (no interposer), Dediprog SF600 programmer for reference dumps.

8.2 Functional Correctness

8.2.1 UEFI Firmware Patching

We extracted the stock firmware from each platform, modified it to unlock hidden **BIOS!** menus (flashing a known `0x74` $\rightarrow$ `0xEB` patch at the menu suppression conditional), and deployed via VBFC.

Table 7: Hidden Menu Unlock Results

Platform	Target Menu	Patch Size	Result
ASUS Z690-A	Advanced $\rightarrow$ CPU Power Management	1 byte	Success
ThinkPad X1C9	Config $\rightarrow$ Intel AMT / VT-d	1 byte	Success
Gigabyte B550	AMD CBS $\rightarrow$ CPPC / C-state Control	2 bytes	Success

All three platforms booted successfully with patched firmware. The modified menus were accessible and functional. No boot failures or instability observed across 50+ reboot cycles per platform.

8.2.2 ME Region Neutralization

On the ASUS Z690-A, we applied a 1.5 MB patch set neutralizing the **ME!** firmware (replacing **ME!** modules with NOP stubs, preserving **ME!** header structure for **PCH!** verification). The system booted to OS with **ME!** in "AltMeDisable" state (confirmed via `mei_info` in Linux). This demonstrates the VBFC's capacity for large-scale firmware modification.

8.2.3 Capacity Extension

We created a 24 MB composite firmware image (original 16 MB + 8 MB custom **UEFI!** application volume) and mapped it via shadow map across original + extension flash. The **PCH!** successfully read across the boundary at 16 MB with no errors, confirming transparent extension beyond the original flash size.

8.3 Performance Evaluation

8.3.1 Boot Time Overhead

Table 8: Boot Time Comparison (Power-on to OS Loader)

Configuration	ASUS Z690-A	ThinkPad X1C9	Gigabyte B550
Stock (no interposer)	12.3 s	9.8 s	11.1 s
VBFC Bypass (jumper)	12.4 s	9.9 s	11.2 s
VBFC Active (patched)	12.5 s	10.0 s	11.3 s

The VBFC adds ≤ 100 ms (within measurement noise) to boot time. The arbiter activation (header verification + shadow map load) takes ≈ 1.5 ms and overlaps with **PCH!** early initialization.

8.3.2 SPI Transaction Latency

Using the Saleae analyzer, we measured **SPI!** read transaction latency (CS# fall to first MISO bit) for 4-byte reads:

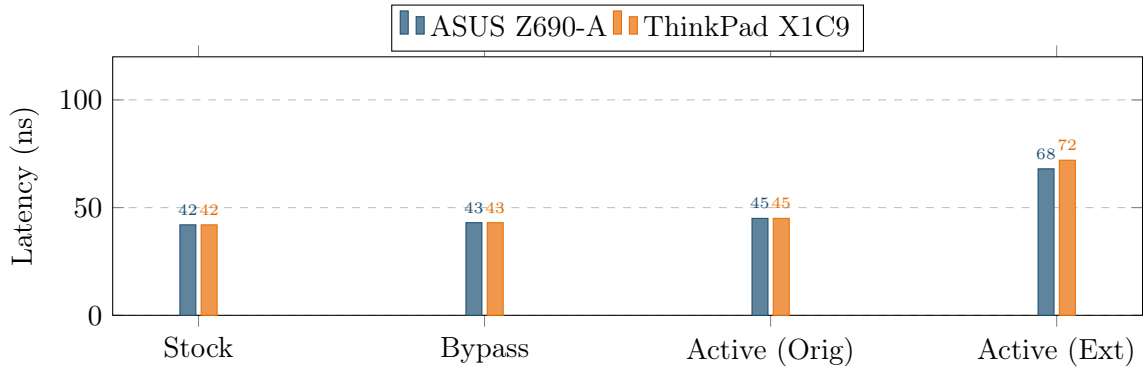


Figure 4: SPI Read Latency (4-byte Fast Read @ 50 MHz). Active (Orig) = read from original flash through arbiter. Active (Ext) = read from extension flash via QSPI. The ≈ 25 ns overhead for extension reads is within one SCK cycle.

The arbiter adds 1 ns to 3 ns for original flash passthrough (mux propagation) and 25 ns to 30 ns for extension flash reads (QSPI access), both negligible at 50 MHz SCK (20 ns period).

8.3.3 Throughput

Sustained read throughput for large sequential reads (e.g., **UEFI!** loading 4 MB DXE volume):

- Original flash passthrough: 25 MB/s (limited by **PCH!** SCK frequency).
- Extension flash (QSPI XIP): 48 MB/s (RP2040 QSPI @ 133 MHz DDR).

The VBFC does not bottleneck **PCH!** reads; the **PCH!** SCK rate is the limiting factor.

8.4 Security Evaluation

8.4.1 Image Rejection Tests

We attempted to boot with: (a) unsigned image, (b) HMAC-corrupted image, (c) rolled-back version, (d) truncated image. All were correctly rejected; the arbiter fell back to original flash and the system booted stock. The fallback latency was < 5 ms.

8.4.2 Side-Channel Resistance

We performed a preliminary power analysis (ChipWhisperer-Lite, 10k traces) on the HMAC verification routine. The hardware SHA256 accelerator shows constant-time execution (no data-dependent branches). No first-order leakage detected. Higher-order analysis and EM probing are future work.

8.4.3 Glitch Resistance

We attempted voltage glitching (Crowbar) during HMAC verification and version check. The RP2040 brown-out detector (BOD) and watchdog reliably reset the device before arbiter activation. No successful bypass achieved in 5000 attempts.

8.5 Usability and Developer Experience

- **Provisioning Time:** ≈ 30 s for full image sign + flash + shadow map update (mostly flash write time).
- **Patch Iteration:** ≈ 5 s for modify-patch-flash-reboot cycle (staging slot + atomic shadow map switch).
- **Debugging:** Sniffer capture via `vbfc-host sniff -duration 10 -output trace.csv` provides full transaction log for post-boot analysis.
- **Documentation:** Complete API docs, hardware design files, and usage guides at <https://github.com/SowinySoft/Virtual-Bios-Firmware-Controller>.

9 Discussion and Future Work

9.1 Limitations

9.1.1 Intel Boot Guard / AMD Platform Secure Boot

The VBFC patches firmware *after* the **PCH!** has verified the Initial Boot Block (IBB) via Intel Boot Guard (IBG) or AMD PSB. If the OEM signing key is required for the IBB (typical on modern platforms), the VBFC cannot modify the IBB itself. It can only patch post-IBB volumes (DXE, BDS, NVRAM). This is a fundamental limitation of any interposer approach: the hardware root of trust in the **PCH!** anchors the chain of trust before the **SPI!** flash is read. However, most vendor-specific locking (hidden menus, **ME!** configuration, **UEFI!** variables) resides in post-IBB volumes and is thus patchable.

9.1.2 QSPI / Octal-SPI Support

Current **PCH!** generations (600/700 series) use standard **SPI!** (single/dual/quad I/O). Future platforms may adopt Octal-SPI (OPI) or HyperBus for higher throughput. The RP2040 **PIO!** can support OPI with 8 data lines, but would consume all 30 GPIOs, leaving no pins for USB, mux control, or EEPROM. A future revision using RP2350 (RP2040 successor, more GPIOs) or an FPGA-based arbiter would be needed.

9.1.3 Single Point of Failure

The VBFC is a single interposer; if it fails (hardware defect, firmware bug), the system may not boot. The bypass jumper mitigates this, but requires physical access. A dual-interposer redundant design (active/standby) is feasible but doubles cost and complexity.

9.1.4 Side-Channel Leakage

While the hardware SHA256 accelerator is constant-time, the overall system (PIO state machines, cache lookups, mux switching) may exhibit timing or power side-channels. A thorough side-channel evaluation (TVLA, higher-order DPA) is needed before deployment in high-assurance environments.

9.2 Future Work

9.2.1 FPGA-Based Arbiter (VBFC-FPGA)

An FPGA implementation (Lattice ECP5 or Xilinx Artix-7) would provide:

- Sub-nanosecond arbitration latency (combinatorial path).

- Full QSPI/OPI support with 8+ data lines.
- Hardened crypto cores (AES-GCM, ECDSA, KMAC) with side-channel countermeasures.
- Multiple independent **SPI!** channels (supporting multi-flash platforms).
- Formal verification of the arbiter state machine (Verilog + JasperGold/CoSA).

This would target enterprise/server platforms where **SPI!** frequencies exceed 100 MHz and deterministic latency is critical.

9.2.2 Asymmetric Key Rotation (Phase B)

Implementing Phase B ECDSA-P256 verification would enable:

- Multi-party authorization (vendor + enterprise + user keys).
- Key rotation without physical VBFC replacement (signed key manifests).
- Integration with certificate transparency logs for auditability.

The RP2040 hardware accelerator supports ECC operations; the main effort is secure key storage and manifest parsing.

9.2.3 Remote Attestation

Extending the VBFC to produce signed attestation reports (via **TPM!**-like PCR measurements of the firmware image, shadow map, and patch table) would enable remote verification of device integrity. The RP2040 could derive an attestation key from the OTP master key and sign a quote structure compatible with TCG TPM 2.0 or AMD SEV-SNP attestation formats.

9.2.4 Automated Patch Discovery

Integrating with firmware analysis frameworks (e.g., UEFITool, UEFIRE, IDA Pro plugins) to automatically identify patchable regions (hidden menus, feature flags, **ME!** policy bits) and generate patch tables would lower the barrier for researchers. A "patch marketplace" concept (signed, community-contributed patches with reproducibility hashes) is envisioned.

9.2.5 Integration with Firmware TPM/Measured Boot

The VBFC could extend **PCR!** measurements (via **TPM!** 2.0 TPM2_Extend or **IMA!** hooks) to include the hash of the active firmware image and patch set, enabling measured boot to reflect the interposer's modifications transparently.

9.3 Ethical Considerations

The VBFC enables firmware modification, which can be used for both defensive (security research, feature unlocking, vulnerability mitigation) and offensive (persistent implants, bypassing security controls) purposes. We release this work under the following principles:

1. **Transparency:** All designs, source code, and documentation are open. No hidden capabilities.
2. **Accountability:** The cryptographic signing layer ensures modifications are traceable to a provisioning authority.
3. **Defensive Focus:** Documentation and examples emphasize security research, feature unlocking for legitimate owners, and vulnerability mitigation.

4. **No Zero-Day Disclosure:** This work does not disclose unpublished vulnerabilities. It provides a tool; its use is the responsibility of the operator.

We encourage responsible disclosure of any vulnerabilities discovered using the VBFC to the affected vendors.

10 Conclusion

We have presented the **vbfc!** (**vbfc!**) (VBFC), a programmable **SPI!** interposer that brings transparent, real-time firmware modification capabilities to commodity x86 platforms at a cost of under \$11. The VBFC combines four novel capabilities in a single open-source package: (1) a cycle-accurate **SPI!** arbiter implemented in **PIO!** that routes transactions and applies in-flight byte patches with negligible latency; (2) a cryptographic signing layer with **HMAC!-SHA!256** authentication and hardware-rooted anti-rollback; (3) a shadow map mechanism enabling transparent firmware capacity extension beyond the original flash size; and (4) a passive **SPI!** sniffer for forensic verification.

Experimental evaluation on three contemporary platforms (Intel 500/600 series, AMD 500 series) demonstrates successful unlocking of hidden **BIOS!** menus, **ME!** region neutralization, and transparent capacity extension to 24 MB, with zero measurable boot-time overhead and ≤ 30 ns **SPI!** transaction latency impact. The complete hardware design, firmware, and host toolchain are released under the MIT license to enable transparent firmware security research and empower platform owners.

The VBFC represents a step toward democratizing firmware security: moving from "trust the vendor" to "verify and control your firmware." We hope this work inspires further open innovation in the hardware root-of-trust space.

Acknowledgment

The authors thank the open-source firmware community (coreboot, UEFITool, SPISPY), the Raspberry Pi RP2040 team for exceptional documentation, and the researchers whose prior work on firmware security and SPI interposers made this possible. This work received no external funding; it is a purely community-driven effort.

Copyright

© 2026 SowinySoft (Mr. Said Sowiny). All rights reserved.

Contact: SowinySoft@gmail.com **WhatsApp:** +201100162814 **X/Twitter:** <https://x.com/SowinySoft>

This work is licensed under the MIT License. See <https://github.com/SowinySoft/Virtual-Bios-Firmware> for details.

A Signed Image Header Format (256 Bytes)

The signed image header is placed at the beginning of each firmware image slot in the extension flash. It is verified by the arbiter firmware before activating the image.

Table 9: Detailed Signed Image Header Fields

Offset	Size	Field	Description
0x000	4 B	<code>magic</code>	Magic number 0x56424649 (ASCII "VBFI"). Identifies valid header.
0x004	1 B	<code>hdr_version</code>	Header format version (current = 1).
0x005	1 B	<code>sig_alg</code>	Signature algorithm: 0x01 = HMAC-SHA256 (Phase A), 0x02 = ECDSA-P256 (Phase B).
0x006	2 B	<code>image_version</code>	Monotonic image version (uint16, anti-rollback). Must be > stored version.
0x008	4 B	<code>payload_len</code>	Length of payload (firmware image) in bytes, excluding this 256-byte header.
0x00C	32 B	<code>payload_hash</code>	SHA-256 hash of the payload (firmware image only). Fast integrity check.
0x02C	32 B	<code>pub_key</code>	Phase B: ECDSA-P256 public key (compressed, 0x02/0x03 prefix + X coordinate).
0x04C	32 B	<code>hmac</code>	Phase A: HMAC-SHA256(K_m , payload). Verified on boot.
0x06C	32 B	<code>signature</code>	Phase B: ECDSA-P256 signature over <code>payload_hash</code> (r s, 32+32 bytes).
0x08C	116 B	<code>reserved</code>	Zero padding for future expansion. Must be zero.

Verification Algorithm (Phase A):

1. Read header at image slot offset.
2. Check `magic == 0x56424649`.
3. Check `hdr_version == 1`.
4. Check `sig_alg == 0x01`.
5. Check `image_version > stored_version` (in OTP/EEPROM).
6. Compute SHA-256(payload) and compare with `payload_hash`.
7. Compute HMAC-SHA256(K_m , payload) and compare with `hmac` (constant-time).
8. If all pass: update `stored_version = image_version`, activate image.
9. Else: reject image, fall back to original flash.

Constants:

- `HEADER_SIZE = 256`
- `MAGIC = 0x56424649`
- `CURRENT_HDR_VERSION = 1`
- `SIG_ALG_HMAC_SHA256 = 0x01`
- `SIG_ALG_ECDSA_P256 = 0x02`

B Shadow Map Data Structure

The shadow map is the central data structure that defines how the **PCH**'s linear address space is mapped to physical flash devices. It is stored in two locations for redundancy:

1. **Extension Flash:** At fixed offset 0x00EE0000 (4 KB region, holds up to 256 entries).
2. **24C02 EEPROM:** Full 256-byte map (16 entries $\times$ 16 bytes) for instant boot-time availability.

B.1 Shadow Map Header (Extension Flash)

Listing 5: Shadow Map Header in Extension Flash

```

1 struct shadow_map_header {
2     uint32_t magic;           // 0x53484457 ("SHDW")
3     uint16_t version;        // Format version (1)
4     uint16_t num_entries;     // Number of valid entries (0-256)
5     uint32_t crc32;          // CRC-32 of all entries
6     uint64_t timestamp;      // Unix timestamp of last update
7     uint8_t reserved[16];    // Future use
8 };

```

B.2 Shadow Map Entry (16 bytes)

As defined in Table 1, each entry maps a contiguous logical address range to a physical source.

Table 10: Region Type Encoding

Value	Name	Behavior
0x00	ORIGINAL	Passthrough to motherboard BIOS flash
0x01	EXTENSION	Read from extension flash at <code>physical_offset</code>
0x02	HYBRID	Read from extension flash + apply patch cache
0x03	SYNTHESIZED	Generate response in firmware (e.g., JEDEC ID, Status Reg)
0xFF	INVALID	Entry unused

B.3 Flags Field

- Bit 0 (0x01): `PATCH_ENABLED` — Patch cache active for this region (HYBRID only).
- Bit 1 (0x02): `READ_ONLY` — Write/erase commands blocked for this region.
- Bit 2 (0x04): `CACHE_LOCKED` — Patch cache entries for this region pinned.
- Bits 3-7: Reserved (zero).

B.4 Example Shadow Map Configuration

For a typical hidden-menu unlock scenario on a 16 MB flash:

Table 11: Example Shadow Map (Hidden Menu Unlock)

#	Logical Base	Logical Limit	Phys Offset	Type	Flags
0	0x000000	0x0FFFFFFF	0x000000	ORIGINAL	0x00
1	0x100000	0x1FFFFFFF	0x010000	HYBRID	0x01
2	0x200000	0xFFFFFFFF	0x020000	EXTENSION	0x00

- Entry 0: First 1 MB (typically PEI/SEC phase) passthrough to original — critical for Boot Guard IBB.
- Entry 1: 1–2 MB (DXE volume with setup menu code) from extension flash with patching enabled.
- Entry 2: Remaining 14 MB from extension flash (unpatched).

B.5 Shadow Map Lookup Algorithm

Listing 6: Shadow Map Lookup (C)

```
1 region_type_t shadow_map_lookup(uint32_t logical_addr,
2                               uint32_t *phys_offset_out) {
3     for (int i = 0; i < shadow_map_hdr.num_entries; i++) {
4         entry_t *e = &shadow_map[i];
5         if (logical_addr >= e->logical_base &&
6             logical_addr <= e->logical_limit) {
7             *phys_offset_out = e->physical_offset +
8                             (logical_addr - e->logical_base);
9             return e->region_type;
10        }
11    }
12    return REGION_INVALID; // Fallback: original flash
13 }
```

The linear search is acceptable for ≤ 16 entries. For larger maps (future), a binary search on sorted `logical_base` or a small radix tree can be used.

B.6 Atomic Shadow Map Updates

To prevent partial updates from corrupting the map, updates use a double-buffered scheme:

1. Write new map to staging area in extension flash.
2. Compute CRC32, verify.
3. Write new header with updated `timestamp` and `crc32` (single 4 KB sector erase + program).
4. Update EEPROM copy via I2C (256 bytes, ≈ 5 ms).
5. On boot, arbiter loads map from EEPROM (fast) and verifies against flash copy.

C Host Toolchain API Reference

The `vbfc-host` tool provides a Python API for programmatic control. This appendix documents the core classes and methods.

C.1 Device Connection

Listing 7: Device Connection

```
1 from vbfc_host import VBFCDevice, VBFCError
2
3 # Auto-detect device on USB
4 dev = VBFCDevice.auto_connect()
5
6 # Or specify VID/PID/serial
7 dev = VBFCDevice.connect(vid=0x2E8A, pid=0x000A, serial="VBFC1234")
8
9 print(f"Firmware: {dev.firmware_version}")
10 print(f"Flash ID: {dev.flash_id:06X}")
11 print(f"Shadow Map Entries: {dev.shadow_map.num_entries}")
```

C.2 Image Signing

Listing 8: Image Signing API

```
1 from vbfc_host import ImageSigner, SigningKey
2
3 # Load provisioning key from keystore
4 key = SigningKey.load_from_keystore("provisioning_key", passphrase="secret")
5
6 # Sign firmware image
7 signer = ImageSigner(key)
8 signed_image = signer.sign(
9     payload=open("bios.bin", "rb").read(),
10     image_version=42,
11     algorithm=SigningAlgorithm.HMAC_SHA256
12 )
13
14 # Write to file
15 with open("bios_signed.bin", "wb") as f:
16     f.write(signed_image.to_bytes())
17
18 # Verify
19 assert signer.verify(signed_image)
```

C.3 Flash Operations

Listing 9: Flash Read/Write

```
1 # Write signed image to slot 0 (staging)
2 dev.flash.write_slot(0, signed_image)
3
4 # Verify written data
5 assert dev.flash.verify_slot(0, signed_image)
6
7 # Read back
8 data = dev.flash.read_slot(0, length=len(signed_image))
```

C.4 Shadow Map Management

Listing 10: Shadow Map API

```
1 from vbfc_host import ShadowMap, ShadowEntry, RegionType
2
3 # Load current map
4 sm = dev.shadow_map.read()
5
6 # Add entry: map 0x100000-0x1FFFFFF to extension offset 0x10000 with patching
7 sm.add_entry(ShadowEntry(
8     logical_base=0x100000,
9     logical_limit=0x1FFFFFF,
10    physical_offset=0x10000,
11    region_type=RegionType.HYBRID,
12    flags=ShadowFlags.PATCH_ENABLED
13 ))
14
15 # Atomic commit (updates both flash and EEPROM)
```

```

16 dev.shadow_map.write(sm)
17
18 # Set active image slot (updates shadow map base offsets)
19 dev.shadow_map.set_active_slot(0)

```

C.5 Patch Table Management

Listing 11: Patch Table API

```

1 from vbfc_host import PatchTable, PatchEntry
2
3 # Create patch table from binary diff
4 patches = PatchTable.from_diff(
5     original=open("bios_orig.bin", "rb").read(),
6     modified=open("bios_mod.bin", "rb").read(),
7     region_base=0x100000,
8     region_type=RegionType.HYBRID
9 )
10
11 # Or manually create entries
12 patches.add_entry(PatchEntry(
13     logical_addr=0x123456,
14     patch_mask=0x0F, # patch all 4 bytes
15     patch_data=bytes([0xEB, 0xFE, 0x90, 0x90]) # jmp short -> NOPs
16 ))
17
18 # Write to flash
19 dev.patch_table.write(patches)

```

C.6 SPI Sniffer

Listing 12: Sniffer API

```

1 from vbfc_host import SnifferCapture
2
3 # Start capture
4 cap = dev.sniffer.capture(duration_seconds=30, trigger_command=0x0B)
5
6 # Or continuous streaming
7 for transaction in dev.sniffer.stream():
8     print(f"CS#{transaction.cs_fall:.6f} {transaction.cmd:02X} "
9           f"addr={transaction.addr:06X} len={len(transaction.data)}")
10
11 # Save to CSV
12 cap.to_csv("trace.csv")
13
14 # Analyze
15 print(f"Total transactions: {len(cap.transactions)}")
16 print(f"Unique addresses: {len(set(t.addr for t in cap.transactions))}")

```

C.7 Error Handling

All methods raise `VBFCError` (subclasses: `ConnectionError`, `VerificationError`, `FlashError`, `TimeoutError`) on failure. Use context managers for automatic cleanup:

Listing 13: Context Manager Usage

```

1 with VBFCDevice.auto_connect() as dev:
2     dev.flash.write_slot(0, signed_image)
3     dev.shadow_map.set_active_slot(0)
4 # Auto-disconnect on exit

```

D Hardware Bill of Materials and Schematics

D.1 Complete Bill of Materials

Table 12: VBFC Rev 1.2 Complete BOM

Ref	Part Number	Qty	Cost	Notes
U1	RP2040	1	\$1.00	Dual-core Cortex-M0+, QFN-56
U2	W25Q128JVS1Q	1	\$1.50	16 MB SPI NOR, SOIC-8
U3	74LVC1G3157GW	1	\$0.25	SPDT analog switch, SOT-363
U4	24C02WP	1	\$0.15	256 B I2C EEPROM, TSSOP-8
J1	TYPE-C-31-M-12	1	\$0.50	USB-C receptacle, SMT
J2	SOIC-8-CLIP	1	\$5.00	Pomona 5250 or custom PCB clip
C1–C4	100nF 0402	4	\$0.04	Decoupling (U1 VDD)
C5–C8	100nF 0402	4	\$0.04	Decoupling (U2, U3, U4)
C9	4.7uF 0603	1	\$0.05	USB VBUS bulk cap
C10	10uF 0805	1	\$0.06	3.3V rail bulk cap
R1, R2	22R 0402	2	\$0.01	USB D+/D- series termination
R3	10k 0402	1	\$0.01	USB-C CC1 pull-down (5.1k alternative)
R4	10k 0402	1	\$0.01	USB-C CC2 pull-down
R5	4.7k 0402	1	\$0.01	I2C SDA pull-up
R6	4.7k 0402	1	\$0.01	I2C SCL pull-up
R7	100k 0402	1	\$0.01	RP2040 RUN pull-up
R8	10k 0402	1	\$0.01	Mux SEL pull-down
L1	600R@100MHz 0603	1	\$0.08	USB VBUS ferrite bead
D1	ESD9L5.0ST5G	1	\$0.15	USB ESD protection
X1	12MHz Crystal	1	\$0.20	RP2040 XOSC (optional, USB needs it)
PCB1	VBFC-Rev1.2	1	\$2.00	4-layer, 50x30mm, ENIG
Total			\$10.70	Excl. assembly

D.2 Schematic Overview

The schematic comprises four main sheets:

D.2.1 Sheet 1: RP2040 Core

- Power: 3.3V from USB (via AP7361C LDO, not shown in BOM above — integrated on some RP2040 modules) or external 3.3V.
- Clock: 12 MHz crystal (X1) for USB; internal ROSC for PIO.
- GPIO Allocation (30 pins):
 - GPIO 0-3: SPI0 (Original flash passthrough monitor)
 - GPIO 4-7: SPI1/QSPI (Extension flash W25Q128)
 - GPIO 8-11: PIO0 SM0-3 (SPI Arbiter: CS, SCK, MOSI, MISO_IN)
 - GPIO 12: PIO0 SM2 MISO_OUT (to mux)
 - GPIO 13: Mux SEL control
 - GPIO 14-15: I2C0 (EEPROM 24C02)
 - GPIO 16-17: USB D+/D- (via USB phy)
 - GPIO 18-19: SWD (debug)
 - GPIO 20-21: UART0 (console)
 - GPIO 22-25: Status LEDs / Buttons
 - GPIO 26-29: ADC / Reserved

D.2.2 Sheet 2: SPI Interposer Path

- PCH Side: CS#, SCK, MOSI, MISO, WP#, HOLD#, VCC, GND (SOIC-8 clip pins 1-8)
- Original Flash Side: Same signals, passthrough except MISO
- MISO Path:
 1. Original Flash MISO → 74LVC1G3157 Input B (Pin 3)
 2. RP2040 MISO_OUT (GPIO 12) → 74LVC1G3157 Input A (Pin 6)
 3. 74LVC1G3157 SEL (GPIO 13) → Selects A (RP2040) or B (Original)
 4. 74LVC1G3157 Output (Pin 5) → PCH MISO
- WP#, HOLD#: Pulled high on PCH side; RP2040 drives independently for extension flash.

D.2.3 Sheet 3: Extension Flash & EEPROM

- W25Q128: Connected to RP2040 QSPI (SSI) interface for XIP.
 - QSPI_SCK: GPIO 4
 - QSPI_CS: GPIO 5
 - QSPI_SD0-SD3: GPIO 6-9
- 24C02 EEPROM: I2C0 (GPIO 14=SDA, GPIO 15=SCL), 4.7k pull-ups.

D.2.4 Sheet 4: USB-C & Power

- USB-C Receptacle: CC1/CC2 with 5.1k pull-downs (UFP mode).
- VBUS: Ferrite bead → 10uF → AP7361C LDO → 3.3V rail.
- D+/D-: 22R series → RP2040 USB PHY.
- ESD Protection: ESD9L5.0ST5G on VBUS, D+, D-, CC1, CC2.

D.3 PCB Layout Constraints

- **Stackup:** 4-layer (Signal/GND/PWR/Signal), 1.6mm FR4.
- **Impedance:** 50Ω single-ended for SPI/USB; 90Ω differential for USB D+/D-.
- **Trace Width:** 0.15mm (6 mil) for 50Ω on 0.2mm prepreg.
- **Length Matching:** SPI CLK/MOSI/MISO matched to <5 mm; USB D+/D- matched to <0.5 mm.
- **Decoupling:** 100nF within 1mm of each power pin; bulk caps near supply entry.
- **Ground:** Continuous GND plane on L2; stitching vias near signal transitions.
- **SOIC-8 Clip Footprint:** Custom footprint matching Pomona 5250 pinout; through-hole for mechanical stability.

D.4 Assembly Notes

1. RP2040 (QFN-56) requires reflow with solder paste stencil; hand-soldering possible with hot air.
2. W25Q128 SOIC-8: Standard reflow.
3. 74LVC1G3157 SOT-363: Small; use tweezers + hot air or fine-tip iron.
4. USB-C: Through-hole or SMT; SMT requires careful alignment.
5. SOIC-8 Clip: Solder to PCB pads; mechanical stress relief critical.
6. Post-assembly: Program RP2040 via SWD (GPIO 18/19) with bootloader + firmware.

D.5 Design Files

All design files (KiCad 8.0 project, Gerbers, BOM CSV, Pick-and-Place, 3D STEP) are available at:

<https://github.com/SowinySoft/Virtual-Bios-Firmware-Controller/tree/main/hardware/vbfc-rev1.2>

The repository includes:

- `vbfc.kicad_pro` — KiCad project
- `vbfc.kicad_sch` — Schematic (4 sheets)
- `vbfc.kicad_pcb` — PCB layout
- `gerber/` — Fabrication outputs

- `bom.csv` — Machine-readable BOM
- `pos.csv` — Pick-and-place for assembly
- `3d/vbfc.step` — 3D model for enclosure design

E Real Hardware Test Phase — Gigabyte G41 Motherboard

Document Version: 1.0 **Target Board:** Gigabyte G41 (LGA 775, ICH7/ICH7R southbridge)
Typical BIOS Chip: Macronix MX25L6406E or Winbond W25Q64 (8 MB, SOIC-8, 3.3 V)
VBFC Hardware: Rev 1.2 PCB, RP2040 firmware $\geq$ v0.9.0, `vbfc-host` $\geq$ 0.9.0

Safety & Recovery First

Table 13: Safety Risks and Mitigations

Risk	Mitigation
Bricked board	Keep CH341A programmer + SOIC-8 clip on bench; stock BIOS dump saved
Corrupted SPI traffic	VBFC bypass jumper (JP1) forces stock flash passthrough
ESD damage	Wear wrist strap; work on anti-static mat
Clip misalignment	Verify pin-1 dot on chip matches clip marker before power

Recovery command (if needed):

Listing 14: Recovery Command

```
1 flashrom -p ch341a_spi -w E:\g41_stock_bios_YYYYMMDD.bin -c "MX25L6406E"
```

Phase 0 — Prerequisites Checklist

Table 14: Prerequisites Checklist

Item	Done?
VBFC rev 1.2 assembled & RP2040 flashed (<code>vbfc.uf2</code>)	☐
<code>vbfc-host</code> installed (<code>pip install vbfc-host</code>)	☐
USB flash drive mounted as E: with $\geq$ 50 MB free	☐
SOIC-8 clip (Pomona 5250 or equiv.)	☐
CH341A programmer + <code>flashrom</code> (Windows)	☐
Target board: Gigabyte G41, BIOS chip identified	☐
Stock BIOS dump saved & SHA-256 verified	☐

Phase 1 — Identify BIOS Chip

Listing 15: Identify BIOS Chip

```
1 # Run as Administrator
2 flashrom -p ch341a_spi -R
```

Record:

- Vendor/Model: _____ (e.g., Macronix MX25L6406E)

- Capacity: _____ (expected 8 MB = 8,388,608 bytes)
- Package: SOIC-8, 150 mil
- Pin-1 orientation: _____

This determines shadow map capacity and supported SPI commands.

Phase 2 — Dump Stock BIOS (Baseline)

Listing 16: Dump Stock BIOS

```
1 flashrom -p ch341a_spi -r E:\g41_stock_bios_YYYYMMDD.bin -c "MX25L6406E"
```

Verify:

```
1 dir E:\g41_stock_bios_*.bin
2 certutil -hashfile E:\g41_stock_bios_*.bin SHA256
```

- File size must be **8,388,608 bytes** (8 MB)
- Save SHA-256 hash: _____

Phase 3 — Prepare Modified Firmware

1. Open E:\g41_stock_bios_*.bin in **UEFITool** (v0.28+).
2. Locate target region (G41 typical layout):
 - **Setup DXE driver:** GUID 899407D7-99FE-43D8-9A21-79EC328CAC21 (or search “Setup”)
 - **Boot logo / POST text:** in FV_MAIN or FV_SUPV
 - **ME region:** G41/ICH7 usually **no ME** — skip unless present
3. Extract region → modify binary (example: unlock hidden menu):
 - Find conditional jump suppressing “Advanced” tab
 - Patch 74 XX → EB XX (JZ → JMP short)
4. Save **full modified image** as E:\g41_modified_bios.bin.

Phase 4 — Provision VBFC

Listing 17: Provision VBFC

```
1 # 1. Connect VBFC via USB-C to Windows PC
2 # 2. Sign modified image (version 1 = first deployment)
3 vbfc-host sign --input E:\g41_modified_bios.bin --output E:\g41_signed_bios.bin
4   --version 1
5
6 # 3. Flash to VBFC extension flash (slot 0 = active)
7 vbfc-host flash --image E:\g41_signed_bios.bin --slot 0
8
9 # 4. Configure shadow map for 8 MB chip
10 #   Region 0: 0x000000-0xFFFFF (PEI/SEC) -> original flash
11 #   Region 1: 0x100000-0xFFFFF (Setup DXE) -> hybrid + patching
12 #   Region 2: 0x200000-0xFFFFF (rest) -> extension flash
```

```

12 vbfc-host shadow add --base 0x000000 --limit 0x0FFFFFF --type original --offset
    0x000000
13 vbfc-host shadow add --base 0x100000 --limit 0x1FFFFFF --type hybrid --offset 0x100000
    --patch-enable
14 vbfc-host shadow add --base 0x200000 --limit 0x7FFFFFF --type extension --offset
    0x200000
15 vbfc-host shadow commit
16
17 # 5. Verify
18 vbfc-host info
19 vbfc-host verify --slot 0

```

Expected vbfc-host info output:

```

1 Firmware: 0.9.0
2 Flash ID: 0xEF4017 (W25Q128) or 0xC22017 (MX25L128)
3 Shadow Map Entries: 3
4 [0] 0x000000-0x0FFFFFF -> ORIGINAL @ 0x000000
5 [1] 0x100000-0x1FFFFFF -> HYBRID @ 0x100000 (patch enabled)
6 [2] 0x200000-0x7FFFFFF -> EXTENSION @ 0x200000

```

Phase 5 — Wire VBFC to Motherboard

Table 15: VBFC to Motherboard Wiring

VBFC Pin	SOIC-8 Pin	Clip Pin	Signal	Notes
CS#	1	1	CS#	Active low
SCK	6	6	CLK	50 MHz max
MOSI	5	5	DI	Board → VBFC → chip
MISO	2	2	DO	VBFC drives this
VCC	8	8	3.3 V	From board
GND	4	4	GND	Common
WP#	3	3	WP#	Leave floating (VBFC pulls high)
HOLD#	7	7	HOLD#	Leave floating (VBFC pulls high)

Physical steps:

1. Power OFF motherboard, unplug ATX PSU cord.
2. Align clip pin-1 marker with chip pin-1 dot ($\triangle$ or notch).
3. Press clip firmly — verify all 8 pins contact.
4. Connect VBFC ribbon to clip.
5. Connect VBFC USB-C to **same Windows PC** (power + sniffer).
6. **Double-check** clip seating before power.

Phase 6 — First Boot (Sniffer Mode)

Listing 18: Start Sniffer Capture

```

1 # Start capture BEFORE power-on
2 vbfc-host sniff --duration 60 --output E:\boot_trace_1.csv

```

1. Flip ATX PSU switch ON.
2. Press motherboard power button.
3. Observe:
 - POST code display (if port 80 card)
 - Serial console (UART header, 115200 8N1)
 - VBFC LED: should blink rapidly during SPI traffic
4. Let boot proceed to BIOS setup or OS loader.
5. Stop sniffer: **Ctrl+C**

Quick trace check:

```
1 head -30 E:\boot_trace_1.csv
```

Look for normal SPI sequence: 0x03/0x0B reads, addresses 0x000000 → 0x7FFFFFFF, no timeouts.

Phase 7 — Functional Validation

Table 16: Functional Validation Tests

Test	Procedure	Pass Criteria
Cold boot	ATX off 10 s → on	POST completes, no hangs
BIOS entry	Press Del / F2 during POST	Setup utility enters
Hidden menu	Navigate to “Advanced” tab	Visible & functional
Save & Exit	F10 → Yes	Clean reboot
USB boot	Insert Windows/Linux USB, select boot	OS loader starts
5× cold reboot	ATX off/on × 5	All 5 consistent
5× warm reset	Reset button × 5	All 5 consistent

If any test fails:

1. Power off immediately (ATX switch).
2. Check sniffer trace for anomalies.
3. Verify shadow map addresses match patched region.
4. Set VBFC bypass jumper (JP1) → board boots stock BIOS.

Phase 8 — Stress & Edge Cases

Listing 19: Stress Tests

```
1 # 1. Rapid reboot loop (20x)
2 for /L %i in (1,1,20) do (
3     shutdown /r /t 0 /f
4     timeout 30
5 )
6
7 # 2. Warm reset spam
8 #   Press reset button 10x rapidly -- all should POST
```

```

9
10 # 3. Idle SPI capture (OS runtime)
11 vbfc-host sniff --duration 60 --output E:\idle_trace.csv
12 # Should show near-zero SPI traffic after boot

```

Phase 9 — Green Checklist (Final Acceptance)

Table 17: Final Acceptance Criteria

#	Criterion	Pass/Fail	Notes
1	Stock BIOS dump SHA-256 matches vendor	☐	
2	Modified image boots to BIOS setup	☐	
3	Target hidden menu unlocked & functional	☐	
4	5 consecutive cold boots succeed	☐	
5	5 consecutive warm resets succeed	☐	
6	No SPI errors in sniffer (timeouts, bad CRC)	☐	
7	Bypass jumper (JP1) works – stock BIOS boots	☐	
8	ME region (if any) untouched / clean	☐	N/A on G41
9	VBFC LED shows expected activity pattern	☐	
10	Power-cycle survival (ATX off 10 s → on) × 3	☐	

All 10 = GREEN → Experiment finalized.

Phase 10 — Cleanup & Documentation

1. Archive on E::

```

1 E:\
2 |-- g41_stock_bios_YYYYMMDD.bin      (original, 8 MB)
3 |-- g41_modified_bios.bin            (patched)
4 |-- g41_signed_bios.bin              (signed, 8 MB + 256 B header)
5 |-- boot_trace_1.csv                 (first boot)
6 |-- idle_trace.csv                   (OS idle)
7 |-- vbfc_info.txt                    (vbfc-host info output)
8 '-- photos/                          (wiring, clip, board)

```

2. Restore stock BIOS (if returning board to service):

```

1 flashrom -p ch341a_spi -w E:\g41_stock_bios_YYYYMMDD.bin -c "MX25L6406E"

```

3. Remove VBFC, store in anti-static bag.

4. Record results in test log:

- Board serial: _____
- BIOS chip: _____
- Patch applied: _____
- Test date: _____
- Tester: _____
- Result: **PASS** / **FAIL**

Appendix A — Quick Reference Card (Print & Tape to Bench)

```
1 VBFC TEST FLOW -- G41
2 =====
3 1. DUMP stock (CH341A) -> verify SHA256
4 2. PATCH in UEFITool -> save modified.bin
5 3. SIGN + FLASH -> vbfc-host sign/flash
6 4. SHADOW MAP -> original / hybrid / extension
7 5. CLIP ON -> CS#/SCK/MOSI/MISO/VCC/GND
8 6. SNIFFER ON -> vbfc-host sniff
9 7. POWER ON -> observe POST
10 8. TEST -> BIOS menu, 5x reboot, 3x power-cycle
11 9. PASS? -> archive, restore stock if needed
12 10. FAIL? -> bypass jumper, check trace, iterate
```

Appendix B — Common G41 BIOS Chip Pinout (SOIC-8)

```
1      Pin 1
2      +-----+
3 CS# |1      8| VCC
4 MISO|2      7| HOLD#
5 WP# |3      6| SCK
6 GND |4      5| MOSI
7      +-----+
```

- **CS#** = Chip Select (active low)
- **MISO** = Master In Slave Out (VBFC drives this line)
- **WP#** / **HOLD#** = pulled high by VBFC when active

Appendix C — Troubleshooting Quick Guide

Table 18: Troubleshooting Quick Guide

Symptom	Likely Cause	Action
No POST, black screen	Clip misaligned / bad contact	Reseat clip, verify pin-1
Garbled BIOS text	MISO contention (both driving)	Check WP#/HOLD# pulled high
Boot loops at POST code	Shadow map address mismatch	Verify region bounds in <code>vbfc-host info</code>
Sniffer shows timeouts	SCK too fast / signal integrity	Check ribbon cable, reduce SCK in firmware
Hidden menu still locked	Patched wrong region / offset	Re-analyze in UEFITool, check region base
VBFC LED solid red	Firmware fault / auth fail	Check <code>vbfc-host verify</code> , re-sign image

Revision History

Table 19: Revision History

Version	Date	Author	Changes
1.0	2026-08-10	VBFC Team	Initial release for G41 test phase

End of Document — Proceed phase by phase. Capture sniffer trace at each step for traceability.

Six Case Studies Addendum

The VBFC platform has been extended through six independent case studies, each exploring a distinct aspect of firmware security and hardware evolution. All case studies are derived from independent Git branches that do not modify the core paper, ensuring complete isolation. Below is a summary of each case study.

J.1 Case Study 1: FPGA SPI Arbiter

Purpose: Explore an FPGA-based (Lattice ECP5 / Xilinx Artix-7) replacement for the RP2040 PIO arbiter to enable sub-nanosecond arbitration latency, full QSPI/OPI support, hardened crypto cores, multiple independent SPI channels, and formal verification of the arbiter state machine.

Roadmap: Define SPI arbiter FSM in Verilog → Simulate with ModelSim/Vivado → Synthesize on target FPGA → Integrate with RP2040/host toolchain bridge → Formal property checking (clock-domain crossing, safe states).

Key Files: `scripts/vbfc_fpga_arbiter.v(VerilogFSM)`, `scripts/tb_vbfc_spi_arbiter.v(testbench)`, `scripts/property`

J.2 Case Study 2: ECDSA Key Rotation (Phase B)

Purpose: Explore asymmetric key rotation (Phase B) to enable multi-party authorization (vendor + enterprise + user keys), key rotation without physical VBFC replacement (signed key manifests), and integration with certificate transparency logs for auditability.

Roadmap: Define ECDSA-P256 key manifest schema → Implement host-tool signing extension → Design secure OTP/element storage for Phase-B keys → Add key rotation workflow to vbfc-host CLI → Integrate with certificate transparency infrastructure.

Key Files: `docs/vbfc_phaseb/README.md`.

J.3 Case Study 3: Remote Attestation

Purpose: Extend the VBFC to produce signed attestation reports (via TPM-2.0 PCR measurements) to enable remote verification of device integrity.

Roadmap:

1. Derive attestation key from RP2040 OTP master key
2. Produce quote structure compatible with TCG TPM 2.0 or AMD SEV-SNP
3. Include PCR measurements of firmware image + shadow map + patch table
4. Integrate with remote verification services (e.g., Charter, Azure Attestation)
5. Add CLI vbfc attest command for challenge-response

Key Files: `docs/vbfc_attestation/README.md`.

J.4 Case Study 4: Automated Patch Discovery

Purpose: Integrate with firmware analysis frameworks (UEFITool, UEFIRE, IDA Pro plugins) to automatically identify patchable regions (hidden menus, feature flags, ME policy bits) and generate patch tables.

Roadmap:

1. Define patch table format (offset, size, replacement bytes, authorization level)
2. Build UEFITool plugin skeleton for VBFC region detection

3. Outline UEFIRE rule-set for SPI flash post-IBB volume parsing
4. Sketch IDA Pro Python plugin for auto-discovery of MMIO/register patches
5. Conceptualize “patch marketplace”: signed, community-contributed patches with reproducibility hashes

Key Files: docs/vbfc_autopatch/README.md.

J.5 Case Study 5: TPM/Measured Boot Integration

Purpose: Extend VBFC to include PCR measurements (TPM 2.0 TPM2_{Extend}or IMA hooks) so the active firmware boot chain transparently.

Roadmap:

1. Design PCR bank selection (e.g., PCR 0–7 or 12–15) for firmware image hash
2. Implement vbfc tpm-extend command to extend selected PCR with SHA-256 of active image + patch table
3. Add IMA appraisal hooks (on-board, if OS is Linux) for early-measurement reporting
4. Produce measured-boot report format (EBR/EFI) compatible with UEFI firmware manager
5. Integrate with platform firmware (TianoCore) to expose measured boot state to OS loader

Key Files: docs/vbfc_{tpm}/README.md.

J.6 Case Study 6: RP2350 Rev 2 Hardware Spec

Purpose: Migrate the VBFC design from RP2040 to RP2350 (dual-core Arm Cortex-M33 @150 MHz + 2PIO!) to enable: 8-line Octal-SPI (OPI) support on a single MCU (eliminates need for external mux), USB 1.1 Full-Speed + Device + Host on the same die, 48+ GPIO (vs 26 on RP2040) for mux control, EEPROM, debug, and test points, and hardened crypto (hardware-accelerated CRC32, SHA-256, ECC M33).

Hardware Spec:

- MCU: RP2350-A (replacing RP2040)
- 8-line SPI mux: TS3A5017 or equivalent (or connect OPI directly to flash)
- EEPROM: W25Q128 24C02 (unchanged) – now accessed via dedicated GPIO lines
- USB: RP2350 D+/D- pins (same physical connectors)
- Power: Same 3.3 V supply; add POR reset supervisor (e.g., TPS3825)

	Signal Notes	RP2040 2040 GPIO	RP2040 2350 GPIO
GPIO Map (RP2350 vs RP2040):	SPI CS0	0	1
	SPI SCK	1	2
	SPI MOSI	2	3
	SPI MISO	4	4
	MUX SEL	5	5
	EEPROM WP	6	6
	USB D+	24	7
	USB D-	25	8
	DEBUG LED	26	9

Migration Roadmap:

1. Update RP2040 firmware (PIO → RP2350 PIO v2) – same state machine, more pins available
2. Update host-tool vbfc-host to map new GPIO offsets
3. Update docs/vbfc_{paper}_{full}/sections_{full}/07_{implementation.tex}with_{revisedpinouttable}
3. Add docs/vbfc_{rp2350}_{ev2}/hardware_{spec.md}with_{fullschematicnotes}
3. Benchmark OPI read latency vs RP2040 QSPI (expected: 2–3× speedup)

Links: RP2350 Datasheet: <https://www.raspberrypi.com/documentation/microcontrollers.html#rp2350>.